

TRAVEL MANAGEMENT SYSTEM



1. Project Introduction

The Travel Management System (TMS) is a Java-based desktop application designed to manage travel bookings efficiently.

The system allows users to create, customize, and manage travel bookings by selecting different travel services such as transportation, insurance, meals, and tour guides.

The main objective of this project is to demonstrate the use of Object-Oriented Programming (OOP) principles and several Design Patterns to build a scalable, maintainable, and well-structured software system.

2. System Architecture Overview

The project follows a **layered architecture** and is organized into the following main packages:

- **tms.ui** → User Interface (GUI using Java Swing)
- **tms.model** → Core business models
- **tms.manager** → Business logic and control layer
- **tms.patterns** → Design pattern implementations
- **tms.persistence** → Data access and storage handling

This structure improves separation of concerns and makes the system easier to extend and maintain.

3. Class Descriptions

3.1 Model Classes

Booking

- **Represents a travel booking.**
- **Contains booking details and a reference to ITravelComponent.**
- **Acts as the base object that can be dynamically enhanced using decorators.**

TravelPackage

- **Represents a complete travel package.**
- **Contains details about accommodation, transportation, meals, insurance, and guide services.**
- **Built using the Builder Pattern.**

3.2 Builder Class

TravelPackageBuilder

The TravelPackageBuilder class is responsible for constructing complex TravelPackage objects step by step.

Purpose:

- **To simplify the creation of travel packages that contain many optional components.**
- **To avoid using large constructors with many parameters.**

Responsibilities:

- **Add transportation**
- **Add accommodation**
- **Add meal plans**
- **Add insurance**
- **Add tour guide services**
- **Build and return the final TravelPackage**

Why it fits the project:

- **Users can customize their travel packages freely.**
- **Different configurations can be created using the same builder.**
- **Improves flexibility and code readability**

3.3 Factory Class

TravelPackageFactory

The **TravelPackageFactory** class is responsible for creating predefined types of travel packages.

Purpose:

- To encapsulate the logic of creating different package types.
- To hide object creation details from the client code.

Examples of packages created:

- Standard Package
- Luxury Package
- Family Package

How it fits the project:

- The user selects a package type.
- The factory creates the appropriate **TravelPackage**.
- Additional customization can then be applied using the **Builder** or **Decorators**.

3.4 Manager Classes

BookingManager

- Responsible for managing booking operations.
- Implements the **Singleton Pattern** to ensure only one instance exists.

- **Uses TravelPackageFactory to create base packages.**
- **Uses TravelPackageBuilder to customize packages.**
- **Applies decorators to add optional services dynamically.**

UserProfileManager

- **Manages user-related operations.**
- **Implemented using the Singleton Pattern to maintain consistent user data across the system.**

3.5 UI Classes (tms.ui)

Examples include:

- **Main_Menu**
- **CreateBookingWindow**
- **BookingsWindow**
- **UsersWindow**
- **PackageWindow**
- **AccommodationWindow**

Purpose:

- **Provide the graphical user interface.**
- **Interact only with manager classes.**
- **Follow MVC principles where the UI acts as the View.**

3.6 Decorator Pattern Classes

ITravelComponent

- **Interface representing the basic travel component.**

Decorator Classes:

- **InsuranceDecorator**
- **MealPlanDecorator**
- **TransportDecorator**
- **GuideDecorator**

Purpose:

- **Dynamically add services and costs to a travel package.**
- **Extend functionality without modifying existing classes.**

- **Allow flexible feature combinations.**

4. Design Patterns Used

This project applies several well-known Design Patterns to ensure flexibility, reusability, and clean separation of responsibilities.

The patterns used are Singleton, Factory, Builder, Decorator, and Proxy, each serving a clear purpose that aligns with the system design and code implementation.



• Singleton Pattern:

Used in:

- BookingManager
- UserProfileManager

Why it was used:

- To ensure that only **one instance** of each manager exists during the application runtime.
- To maintain consistent booking operations and user data across the system.

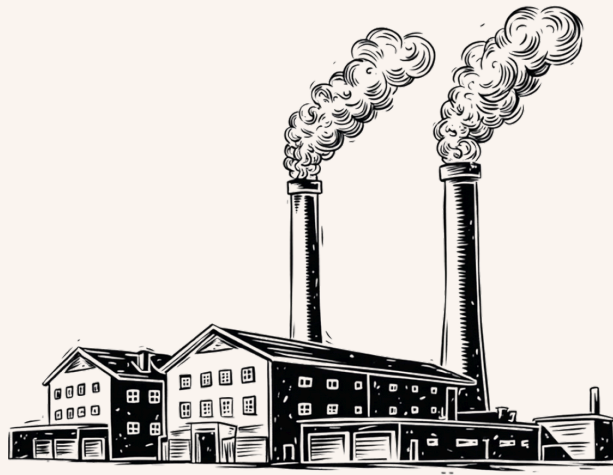
How it is implemented:

Each class has a private static instance.

- A private constructor prevents direct object creation.
- A public getInstance() method provides global access.

How it fits the project:

- BookingManager handles all booking-related operations (flights, accommodations, and packages).
- UserProfileManager manages user details such as preferences, loyalty points, and booking history.
- Using Singleton prevents data inconsistency and duplicated managers.



. Factory Pattern:

Used in:

- **TravelPackageFactory**
- **AccommodationFactory**

Why it was used:

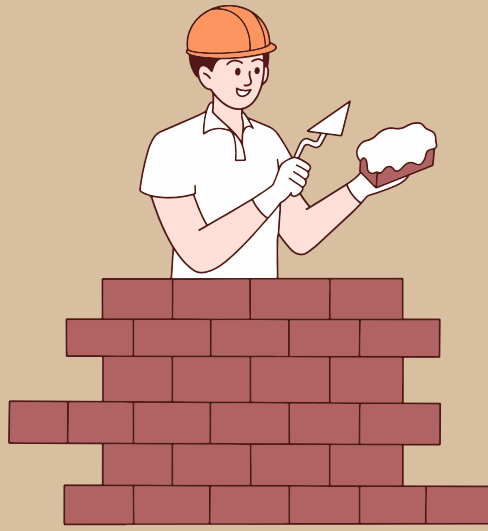
- To centralize and encapsulate object creation logic.
- To hide implementation details from the client code.
- To make the system easy to extend with new types.

How it is implemented:

- **TravelPackageFactory** creates different travel packages (e.g., Luxury, Adventure, Cultural) based on user selection.
- **AccommodationFactory** creates different accommodation types such as:
 - **Hotel**
 - **Hostel**
 - **Resort**

How it fits the project:

- The user selects a package or accommodation type.
- The factory decides which concrete class to instantiate.
- This reduces coupling and simplifies adding new package or accommodation types.



. Builder Pattern:

Used in:

- `TravelPackageBuilder`

Why it was used:

- Travel packages consist of multiple optional attributes such as type, destination, price, and activities.
- Using a single constructor would be complex and hard to maintain.

How it is implemented:

- The builder sets package attributes step by step:
 - `setType()`

- `setDestination()`
- `setPrice()`

The `build()` method returns the final `TravelPackage` object.

How it fits the project:

- Allows creating customized travel packages based on user preferences.

Improves readability and flexibility.

- Works well alongside the Factory pattern, where factories create base objects and builders customize them.



. Decorator Pattern:

Used in:

- ITravelComponent
- TravelPackage
- TravelDecorator
- InsuranceDecorator

Why it was used:

- To add optional services dynamically without modifying existing classes.
- To avoid creating many subclasses for each feature combination.

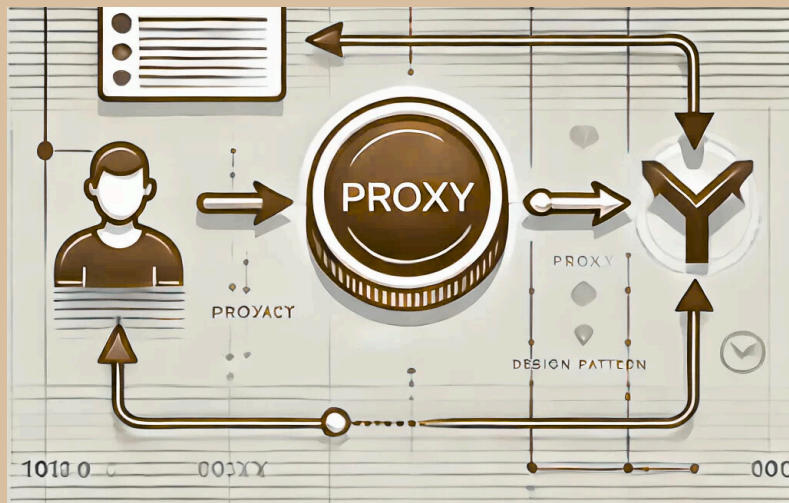
How it is implemented:

- ITravelComponent defines the base interface.
- TravelPackage acts as a concrete component.
- TravelDecorator wraps an ITravelComponent.

- InsuranceDecorator extends functionality by adding extra cost and description.

How it fits the project:

- Optional services (such as insurance) can be added to a travel package at runtime.
- Each decorator increases the total price and updates the description dynamically.
- Keeps the system extensible and flexible.



. Proxy Pattern:

Used in:

- IBookingRepository
- SqlBookingRepository
- BookingRepositoryProxy

Why it was used:

- To control access to the database layer.
- To add an extra level of abstraction between business logic and data storage.

How it is implemented:

- IBookingRepository defines the repository interface.
- SqlBookingRepository provides the real database operations.
- BookingRepositoryProxy acts as an intermediary that forwards requests to the real repository.

How it fits the project:

- The system communicates with the proxy instead of the real repository directly.
- The proxy can be extended later to add logging, validation, or security checks.
- Improves maintainability and control over data access.

. Adapter Pattern (Bonus):

Used in:

- **AccomodationAdapter**
- **ITravelComponent**
- **Accommodation**

Why it was used

The Adapter Pattern was used to allow the existing Accommodation class to work with the ITravelComponent interface without modifying the original class.

In the system, travel services such as insurance and travel packages already implement ITravelComponent.

However, the Accommodation class was designed independently and does not implement this interface.

To integrate accommodation pricing and descriptions into the travel component system, an adapter was required.

How it is used

The AccomodationAdapter class:

- Implements the ITravelComponent interface.
- Wraps an instance of the Accommodation class.
- Converts accommodation-specific data into a format compatible with ITravelComponent.

Specifically:

- **getDescription()** adapts accommodation details (name, location, number of nights) into a unified description.
- **getPrice()** adapts the pricing logic by calculating the total cost based on $\text{pricePerNight} \times \text{numberOfNights}$.

This allows accommodation objects to be treated as regular travel components within the system.

How it fits the project

- **The adapter enables accommodation to be added to bookings using the same interface as other travel components.**
- **It allows the system to apply decorators and pricing logic consistently.**
- **It avoids modifying or duplicating the existing Accommodation class.**
- **It supports the Open–Closed Principle, where classes are open for extension but closed for modification.**

Benefits

- **Seamless integration of Accommodation into the travel component system**
- **No changes required to existing model classes**
- **Improved flexibility and code reuse**
- **Cleaner and more maintainable architecture**

4.5 MVC Concept

The system loosely follows the Model–View–Controller (MVC) architecture:

- **Model: Booking, TravelPackage**
- **View: Swing UI classes**
- **Controller: Manager classes**

5. Conclusion

The Travel Management System effectively demonstrates the use of multiple Design Patterns, including Singleton, Builder, Factory, and Decorator.

These patterns work together to provide:

- **Flexible travel package creation**
- **Clean and maintainable architecture**
- **Easy future extension**
- **Clear separation of responsibilities**

The system design and documentation fully align with the project requirements and academic objectives.

Team Members 😊

Name	ID
Omar Ayman Allam	2201061
Manar Ali Mohamed	2201217
Basmala Ahmed Samir	2201045
Basmala Ahmed Mohamed	2201157
essraa naji Albadry	2201284